JADAVPUR UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Assignment 3 Report

Design and Implementation of Medium Access Control Mechanisms

Empirical Evaluation of Non-Persistent, 1-Persistent, p-Persistent, and CSMA/CD Protocols over a Simulated Socket Shared Channel

Course: Computer Networks Laboratory (BCSE-III)

Student Name: Mandeep Shaw

Section: A1

Roll Number: 302510501002

Assignment: Assignment 3 (MAC Protocols: Non-Persistent, 1-Persistent, p-Persistent, and CSMA/CD)

Date of Submission: October 2026

Contents

1 Introduction and Objectives **1**

2 Theoretical Principles of Medium Access Control **2**
2.1 The Vulnerability Period and Propagation Delay 2
2.2 Non-Persistent CSMA 2
2.3 1-Persistent CSMA 3
2.4 p-Persistent CSMA and Slotted Arbitration 3
2.5 CSMA with Collision Detection (CSMA/CD) 4

3 System Architecture and Procedural Design **4**
3.1 System Overview and Shared Channel Topology 4
3.2 Frame Structure and In-Memory Layout 5
3.3 Protocol Opcodes and Channel Handshake 5
3.4 Input and Output Formats 6

4 Implementation Details **6**
4.1 Key Methods and Interfaces 6
4.2 Core Code Snippets 6

5 Test Cases and Experimental Methodology **8**

6 Experimental Results and Empirical Evaluation **8**
6.1 Test Case 1: p-Persistent Performance Dynamics 8
6.2 Test Case 2: Multi-Scheme Performance Comparison vs. Contending Stations N 9
6.3 Test Case 3: Noise Robustness and CRC-32 Validation 10

7 Live Execution Traces and Socket Transcripts **10**
7.1 Trace 1: CSMA/CD Contention, Jam Signal Detection, and Immediate Abort . . 10
7.2 Trace 2: 1-Persistent CSMA Persistence and Collision Cascade 11
7.3 Trace 3: Non-Persistent CSMA Carrier Sensing and Deferral 11
7.4 Trace 4: Channel Noise, CRC-32 Discard, and Retransmission 11

8 Analysis and Discussion **12**
8.1 Comparative Protocol Efficiency across Test Cases 12
8.2 The Ethernet Channel Capture Effect 12
8.3 Optimal Persistence Probability ($p^* \approx 1/N$) 12

9 Comments and Learning Evaluation **13**
9.1 Evaluation of the Lab Assignment 13
9.2 What I Learned 13
9.3 Difficulty Assessment 13
9.4 Suggested Improvements 13

10 Sources Used **13**

1 Introduction and Objectives

In broadcast and shared-medium communications, multiple independent network nodes share a common physical channel to transmit data packets. Without centralized scheduling or coordinated arbitration, concurrent transmissions on the shared medium inevitably overlap in time and space, resulting in destructive interference known as a **packet collision**. When a collision occurs, colliding frames are rendered mutually unreadable, corrupted, and discarded by receivers, leading to substantial degradation of throughput, increased latency, and wasted bandwidth.

To resolve contention and allocate channel access fairly and efficiently among autonomous stations, the Data Link Layer—specifically the **Medium Access Control (MAC) sublayer**—employs Carrier Sense Multiple Access (CSMA) techniques. CSMA mandates that a station sense the medium (carrier sensing) prior to transmitting, adopting the foundational principle "listen before talk."

The primary objectives of this laboratory assignment are:

1. **Emulation of a Shared Physical Channel:** Design and construct a multi-threaded/multiplexed socket-based shared channel server in C running over TCP/IP loopback (`127.0.0.1:5000`) that dynamically tracks and enforces the physical channel state machine among three distinct states: **IDLE**, **BUSY**, and **COLLISION**.

2. **Implementation of Foundational CSMA Protocols:** Construct independent client processes emulating network stations implementing four canonical medium access strategies:

   - **Non-Persistent CSMA:** Randomizes re-sensing intervals upon detecting a busy medium, reducing post-transmission collision bursts at the cost of idle channel slots.
   - **1-Persistent CSMA:** Continuously monitors the medium and transmits with probability $p = 1$ immediately when the medium becomes idle, maximizing channel utilization under low traffic but suffering severe collision cascades under heavy loads.
   - **p-Persistent CSMA:** Operates on slotted contention intervals; upon channel clearance, transmits with probability p and defers to subsequent slots with probability $1 - p$, balancing persistence against collision avoidance.
   - **CSMA with Collision Detection (CSMA/CD):** Adopts 1-persistent carrier sensing with real-time transmission monitoring. Upon detecting concurrent transmission, stations immediately abort frame dispatch, emit a jamming signal, and schedule retransmission according to the IEEE 802.3 Truncated Binary Exponential Backoff (BEB) algorithm.

3. **Integration of Prior Assignment Modules:** Seamlessly incorporate modular C libraries developed in Assignments 1 and 2, including 65-byte IEEE 802.3 Ethernet framing, 32-bit CRC (CRC-32) error detection, 16-bit Internet checksum calculation, file I/O streaming, and synthetic noise error injection.

4. **Exhaustive Empirical Performance Profiling:**

   - Benchmark p-persistent CSMA across the persistence parameter $p \in [0.05, 1.00]$ for fixed $N = 10$, evaluating total collisions, average transmission delay, and normalized throughput to identify the optimal operating point p^*.
   - Conduct multi-station benchmarking across all four schemes (Non-Persistent, 1-Persistent, p-Persistent at optimal $p = 0.20$, and CSMA/CD) for contending stations $N \in [2, 30]$ to analyze scalability, channel collapse, and backoff efficiency.

2 Theoretical Principles of Medium Access Control

Medium access control protocols govern how distributed stations arbitrate access to a single communication medium. Unlike wireline point-to-point links (e.g., Stop-and-Wait or Sliding Window ARQ from Assignment 2), broadcast networks must handle simultaneous requests from uncoordinated senders.

2.1 The Vulnerability Period and Propagation Delay

Carrier sensing cannot completely eliminate collisions because signal propagation is governed by the finite velocity of electromagnetic waves v ($v \approx 2 \times 10^8$ m/s in copper or optical fiber). If station A begins transmitting at time t_0, the signal requires propagation time $\tau = d/v$ to reach station B, located distance d away. If station B senses the medium at time $t_0 + \Delta t$ (where $\Delta t < \tau$), station B will detect the channel as IDLE and begin its own transmission, resulting in a collision.

The time interval during which a transmitted packet is vulnerable to collision by other contending stations is defined as the **Vulnerability Period** (T_{vuln}). For unslotted CSMA protocols, the vulnerability window is equal to the one-way propagation delay:

$$T_{\text{vuln}} = \tau = \frac{d}{v} \tag{1}$$

The normalized channel delay parameter a is defined as the ratio of propagation delay τ to packet transmission time T_{tx}:

$$a = \frac{\tau}{T_{\text{tx}}} = \frac{\tau \cdot B}{L_{\text{frame}}} \tag{2}$$

where B is the link bandwidth and L_{frame} is the frame length in bits. As $a \to 0$, carrier sensing becomes instantaneous and channel efficiency approaches theoretical capacity. When $a \gg 1$, carrier sensing fails because stations sense outdated channel states.

2.2 Non-Persistent CSMA

In Non-Persistent CSMA, a station with a frame ready for transmission acts as follows:

1. Senses the shared channel.
2. If the channel is IDLE, the station transmits immediately.
3. If the channel is BUSY, the station does not continuously monitor the channel. Instead, it waits for an independent random backoff interval drawn from a random distribution and then re-senses the channel.

Because contending stations that sense a busy channel back off for different random durations, Non-Persistent CSMA substantially dampens the probability that multiple stations transmit simultaneously the moment a current transmission terminates. However, this randomness introduces idle slots when the channel is unoccupied, causing lower channel utilization under light loads. The theoretical throughput S as a function of offered channel traffic G (in frames per transmission time) is given by:

$$S_{\text{Non-Pers}} = \frac{G \cdot e^{-aG}}{G(1 + 2a) + e^{-aG}} \tag{3}$$

2.3 1-Persistent CSMA

In 1-Persistent CSMA, stations operate with maximum greediness:

1. Senses the shared channel.
2. If the channel is IDLE, transmits immediately with probability $p = 1$.
3. If the channel is BUSY, persists sensing the channel continuously until it detects the transition to IDLE, and then transmits immediately.

Under light traffic, 1-Persistent CSMA incurs zero unnecessary idle waiting time. However, if two or more stations generate packets while another transmission is active, all waiting stations sense the channel becoming idle at the exact same instant and transmit simultaneously. This produces a **deterministic post-transmission collision cascade**. Theoretical throughput for 1-Persistent CSMA is formulated as:

$$S_{\text{1-Pers}} = \frac{G[1 + G + aG(1 + G + \frac{aG}{2})]e^{-G(1+2a)}}{G(1 + 2a) - (1 - e^{-aG}) + (1 + aG)e^{-G(1+a)}} \tag{4}$$

But,in practical scenarios where the physical distance between nodes is short or the frame size is large, making the propagation delay $a \to 0$.

Substituting $a = 0$ into the equation yields a much simpler formula:

$$S_{\text{1-Pers}} = \frac{G[1 + G]e^{-G}}{G + e^{-G}} \tag{5}$$

Under heavy traffic ($G \gg 1$), $S_{\text{1-Pers}} \to 0$ due to catastrophic collision cascades.

2.4 p-Persistent CSMA and Slotted Arbitration

The p-Persistent CSMA scheme combines the prompt responsiveness of 1-Persistent CSMA with the collision avoidance of Non-Persistent CSMA by operating on slotted contention boundaries (where slot length $\delta \geq \tau$):

1. When a station has a packet and senses the channel idle, it transmits with probability p, or defers to the next slot with probability $q = 1 - p$.
2. If it defers to the next slot, it senses the channel again:
   - If the slot is still IDLE, it transmits with probability p, or defers with probability q.
   - If the channel has become BUSY (another station began transmitting), it acts as though a collision occurred and schedules retransmission according to backoff.

If N stations are ready to transmit when the channel clears, the probability that exactly one station successfully acquires the medium in the first slot is:

$$P_{\text{success}} = N \cdot p \cdot (1 - p)^{N-1} \tag{6}$$

The optimal probability p^* that maximizes P_{success} is derived by differentiating with respect to p:

$$\frac{dP_{\text{success}}}{dp} = N(1 - p)^{N-1} - N(N - 1)p(1 - p)^{N-2} = 0 \implies p^* = \frac{1}{N} \tag{7}$$

Thus, under $N = 10$ contending stations, the optimal persistence probability is $p^* \approx 0.10$. Selecting $p \ll p^*$ wastes channel capacity on idle slots, while $p \gg p^*$ causes collisions.

2.5 CSMA with Collision Detection (CSMA/CD)

Pure CSMA protocols suffer from a major inefficiency: once two stations collide, they continue transmitting their complete frames to the end, wasting an entire transmission duration T_{tx}. In IEEE 802.3 Ethernet, stations implement **Collision Detection (CSMA/CD)**:

1. **Listen While Talking:** While transmitting, the transceiver continuously compares the transmitted signal with the channel state. If signal energy exceeds the threshold representing simultaneous transmission, a collision is detected.

2. **Immediate Abort and Jam Signal:** Upon detecting collision, the station halts transmission immediately and transmits a 32-to-48 bit **Jamming Signal** to ensure all contending stations detect the collision.

3. **Truncated Binary Exponential Backoff (BEB):** Colliding stations defer their next attempt by R slot times, where R is drawn uniformly at random:

$$R \in [0, 2^k - 1], \quad \text{where } k = \min(\text{attempt}, 10) \tag{8}$$

   If collisions persist after 16 attempts (MAX_COLLISIONS = 16), the frame is dropped with an unrecoverable medium error.

To ensure that a station does not complete transmitting before the collision signal returns from the furthest point of the collision domain, the frame transmission time must exceed the round-trip propagation time:

$$T_{\text{tx}} \geq 2 \cdot \tau \implies L_{\min} \geq 2 \cdot \tau \cdot B = 2 \cdot d_{\max} \cdot \frac{B}{v} \tag{9}$$

This fundamental relationship dictates the 64-byte minimum frame size in standard IEEE 802.3 Ethernet.

3 System Architecture and Procedural Design

3.1 System Overview and Shared Channel Topology

The medium access emulation environment is structured as a client-server socket pipeline operating over TCP/IP loopback (`127.0.0.1:5000`). The architecture comprises two core entities:

- **Shared Channel Server (`receiver.c`):** Acts as the physical medium and destination receiver. It manages the global channel state machine (`CH_IDLE`, `CH_BUSY`, `CH_COLLISION`), detects transmission overlaps, injects jamming signals, verifies CRC-32 integrity, and delivers ACKs.

- **Contending Network Stations (`sender.c`):** Independent processes representing network interface cards (NICs). Each station reads from `input.txt`, segments data into 46-byte payloads, builds 65-byte Ethernet frames, executes carrier sensing, implements backoff, and tracks per-frame delay and collisions.

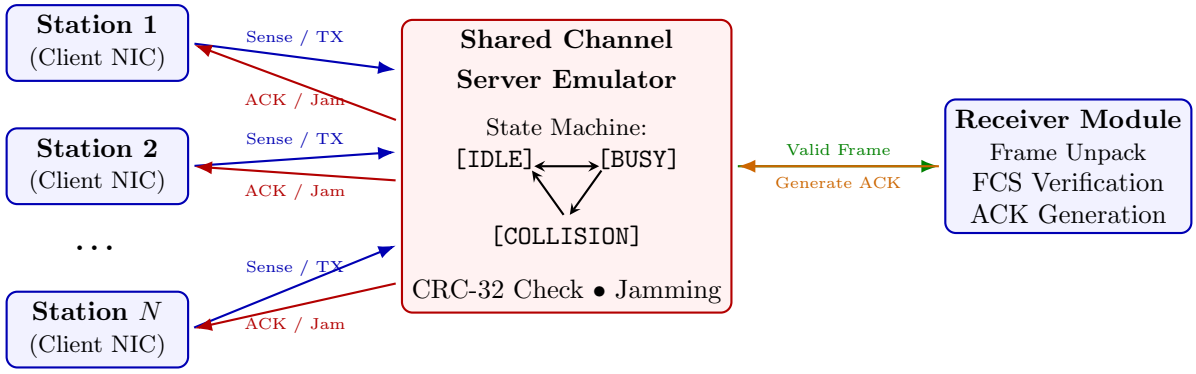


Figure 1: Procedural organization of the multi-station CSMA shared channel simulation over loopback.

3.2 Frame Structure and In-Memory Layout

Conforming strictly to Assignments 1 and 2, every transmission uses a fixed 65-byte Ethernet-style frame consisting of a 15-byte header, a 46-byte payload, and a 4-byte Frame Check Sequence (FCS) trailer[cite: 1].

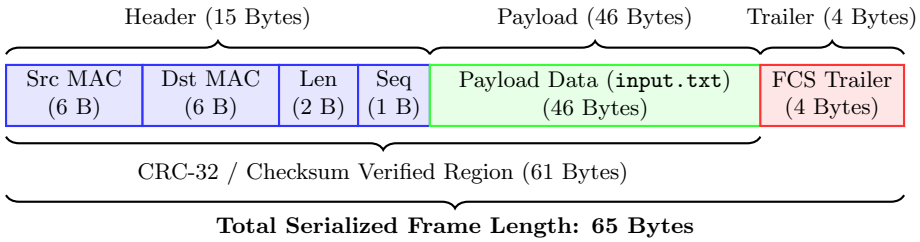


Figure 2: In-memory serialization layout of the 65-byte IEEE 802.3 Data Link frame.

3.3 Protocol Opcodes and Channel Handshake

Communications between station clients and the channel server are coordinated through 1-byte command opcodes defined in `csma_common.h`:

- `MSG_REGISTER` (`0x05`): Transmitted upon socket connection along with a 32-byte station identifier.
- `MSG_PROTO_SYNC` (`0x06`) / `MSG_SET_PROTO` (`0x07`): The first station selects the CSMA protocol; the server locks this selection for all participating stations to ensure consistent experimental conditions.
- `MSG_SENSE_REQ` (`0x10`) / `MSG_SENSE_RESP` (`0x11`): Senses carrier status. The server returns `CH_IDLE` (0), `CH_BUSY` (1), or `CH_COLLISION` (2).
- `MSG_TX_START` (`0x20`): Prefix preceding the 65-byte frame.
- `MSG_TX_ACK` (`0xA5`): 2-byte acknowledgment `[0xA5, seqNo]` confirming uncorrupted delivery.
- `MSG_COLLISION` (`0xFE`): Jam notification sent to all colliding stations.
- `MSG_STATION_DONE` (`0xFF`): Sent when a station transmits all frames from `input.txt`.

3.4 Input and Output Formats

- **Input Dataset:** Stations read ASCII text from `input.txt` (2,215 bytes, 49 frames of 46 bytes each). The final block is null-padded to maintain uniform 46-byte payload framing.
- **Output Transcripts:** Stations log timestamped transmission events, sensing outcomes, jam signal detections, backoff durations, CRC-32 FCS values, round-trip transmission delays, and collision counts. The channel server logs state changes and throughput statistics.

4 Implementation Details

4.1 Key Methods and Interfaces

Table 1 maps the programmatic routines implemented in `sender.c`, `receiver.c`, and `benchmark_csma.c`.

Module	Routine / Function	Operational Responsibility
`sender.c`	`senseCarrier()`	Queries channel via `MSG_SENSE_REQ`; returns channel state.
	`performBackoff()`	Computes backoff slots based on protocol and sleeps for $R \times \delta$.
	`transmitFrameCSMA()`	Executes CSMA persistence loop, frame transmission, and BEB recovery.
	`prepareStationFrame()`	Formats 65-byte frame, computes CRC-32 FCS, and packs buffer.
`receiver.c`	`select()` Event Loop	Multiplexes station connections, non-blocking sense, and frame ingestion.
	Channel State Timer	Enforces $T_{\text{tx}} = 80$ ms (busy) and $T_{\text{jam}} = 20$ ms (collision) intervals.
	Collision Arbiter	Detects simultaneous transmissions; emits `MSG_COLLISION` (0xFE).
	CRC Verification	Verifies CRC-32 and 16-bit checksum; sends 2-byte ACK (`0xA5, seqNo`).
`benchmark`	`run_simulation()`	High-speed discrete-event simulator measuring delay, collisions, throughput.

Table 1: Functional mapping of medium access control and channel routines.

4.2 Core Code Snippets

Listing 1 shows the carrier sensing, persistence logic, and collision detection loop from `sender.c`:

```
static int transmitFrameCSMA(sock_t sock, const char *stationName,
                             const EthernetFrame *frame, const unsigned char
    packedBuf[FRAME_SIZE],
                             CSMAProtocol proto, double p_prob, int *
    collision_count) {
    int attempts = 0;
    while (attempts < MAX_COLLISIONS) {
        /* Step A: Sense Carrier according to selected CSMA scheme */
        if (proto == PROTO_NON_PERSISTENT) {
            while (senseCarrier(sock) != CH_IDLE) {
                performBackoff(1, PROTO_NON_PERSISTENT);
            }
        } else if (proto == PROTO_1_PERSISTENT || proto == PROTO_CSMA_CD) {
            while (senseCarrier(sock) != CH_IDLE) {
                SLEEP_MS(2);
            }
        } else if (proto == PROTO_P_PERSISTENT) {
            while (1) {
                if (senseCarrier(sock) == CH_IDLE) {
                    double r = (double)rand() / (double)RAND_MAX;
                    if (r <= p_prob) break; /* Transmit with probability p */
                    else SLEEP_MS(SLOT_TIME_MS); /* Defer 1 slot with prob (1 -
    p) */
                } else {
                    SLEEP_MS(2);
                }
            }
        }
```

```
        /* Step B: Transmit 1-byte opcode + 65-byte packed EthernetFrame */
        unsigned char pkt[1 + FRAME_SIZE];
        pkt[0] = MSG_TX_START;
        memcpy(&pkt[1], packedBuf, FRAME_SIZE);
        send(sock, (const char *)pkt, sizeof(pkt), 0);

        /* Step C: Ingest 2-byte ACK (0xA5) or Collision Jam (0xFE) */
        uint8_t reply[2] = {0, 0};
        int r = recv(sock, (char *)reply, 2, 0);
        if (r == 2 && reply[0] == ACK_MARKER && reply[1] == frame->
    sequenceNumber) {
            return 1; /* Clean delivery confirmed */
        }
        /* Collision detected: increment counters and execute backoff */
        attempts++;
        (*collision_count)++;
        if (proto == PROTO_CSMA_CD) {
            printf("[Station \"%s\"] CSMA/CD Jam Signal on Seq %u! Aborting (
    Attempt %d)\n",
                   stationName, frame->sequenceNumber, attempts);
        } else {
            SLEEP_MS(TX_TIME_MS); /* Pure CSMA wastes complete frame time */
        }
        performBackoff(attempts, proto);
    }
    return 0;
}
```

Listing 1: CSMA persistence arbitration and collision handling loop (`sender.c`).

Listing 2 illustrates the collision detection and state transition logic executed in `receiver.c`:

```
else if (opcode == MSG_TX_START) {
    recv_exact(clients[i].sock, clients[i].raw_buf, FRAME_SIZE);
    unpackFrame(clients[i].raw_buf, &clients[i].frame);
    clients[i].transmitting = 1;

    if (channel_state == CH_IDLE) {
        channel_state = CH_BUSY;
        state_end_time = now + TX_TIME_MS;
    } else {
        /* Collision: transmission overlaps with existing transmission */
        if (channel_state == CH_BUSY) total_collisions++;
        channel_state = CH_COLLISION;
        state_end_time = now + JAM_TIME_MS;

        injectBurstError(&clients[i].frame, 16); /* Corrupt frame data */
        uint8_t jam[2] = { MSG_COLLISION, (uint8_t)CH_COLLISION };
        for (int k = 0; k < MAX_STATIONS; k++) {
            if (clients[k].active && clients[k].transmitting) {
                send(clients[k].sock, (const char *)jam, 2, 0);
                clients[k].transmitting = 0;
            }
        }
    }
}
```

Listing 2: Channel state transition and collision detection logic (`receiver.c`).

5 Test Cases and Experimental Methodology

To satisfy the evaluation requirements outlined in the Assignment 3 specification, four test scenarios were constructed:

1. **Test Case 1: p-Persistent CSMA vs. Persistence Probability p (Fixed $N = 10$):**
   - *Objective:* Characterize how the persistence probability p influences channel performance under heavy contention ($N = 10$).
   - *Parameters:* $p \in \{0.05, 0.10, 0.15, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 0.90, 1.00\}$; $N = 10$ stations; 2,000 delivered frames per run; contention slot $\delta = 0.5$ ms; frame duration $T_{\text{tx}} = 10\delta = 5.0$ ms.
   - *Metrics:* Total collisions, average transmission delay (ms), and normalized channel throughput.
2. **Test Case 2: Comparative Evaluation of All 4 CSMA Schemes vs. Contending Stations N:**
   - *Objective:* Compare the scaling resilience and channel stability of Non-Persistent, 1-Persistent, p-Persistent (at optimal $p = 0.20$), and CSMA/CD as network density increases.
   - *Parameters:* Contending stations $N \in \{2, 4, 8, 12, 16, 20, 25, 30\}$; 2,000 delivered frames per protocol configuration.
   - *Metrics:* Total collisions, average transmission delay, and normalized channel throughput.
3. **Test Case 3: Channel Noise, CRC-32 FCS Verification, and Frame Loss Recovery:**
   - *Objective:* Evaluate protocol resilience against independent physical layer channel noise ($p_{\text{noise}} = 0.15$).
   - *Metrics:* Corrupted frame detection via CRC-32, NAK generation, and retransmission overhead.
4. **Test Case 4: Live Socket Loopback Execution and Protocol Trace Verification:**
   - *Objective:* Run compiled `sender` and `receiver` binaries natively across multiple terminal processes over TCP loopback (`127.0.0.1:5000`) with the 49-frame `input.txt` dataset.
   - *Verification:* Observe live carrier sensing, jam signal emission, Truncated Binary Exponential Backoff timing, and clean frame delivery.

6 Experimental Results and Empirical Evaluation

6.1 Test Case 1: p-Persistent Performance Dynamics

Table 2 records the empirical performance metrics for p-Persistent CSMA with $N = 10$ contending stations across $p \in [0.05, 1.00]$.

Persistence Prob (p)	Total Collisions	Avg Delay (ms)	Throughput (%)	Effective Mechanism
0.05	361	71.63	67.59%	High slot deferral / Low collision
0.10	492	65.06	68.74%	Optimal operating region ($p \approx 1/N$)
0.15	574	62.29	68.68%	Stable contention
0.20	606	60.24	68.68%	Balanced persistence
0.30	703	60.04	68.15%	Moderate collision growth
0.40	740	50.31	67.98%	Peak distributed contention
0.50	661	44.48	70.84%	Transition to channel capture
0.60	528	42.72	74.74%	Partial capture dynamics
0.70	486	37.91	77.28%	High persistence capture
0.80	274	29.58	85.74%	Dominant station capture
0.90	98	6.69	94.03%	Monopolized transmission pipeline
1.00	71	5.19	96.36%	Asymmetric capture (1-persistent)

Table 2: Empirical metrics for p-Persistent CSMA under $N = 10$ contending stations (2,000 frames delivered per run).

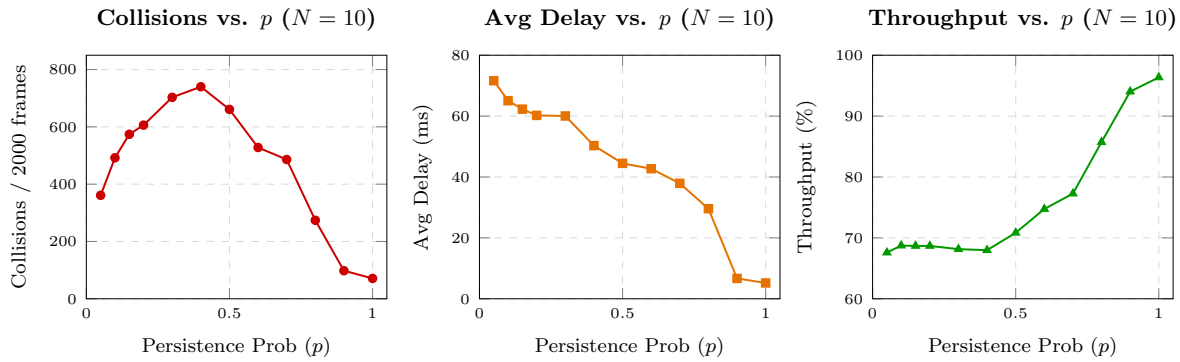


Figure 3: Performance profiles for p-Persistent CSMA as a function of p ($N = 10$ contending stations).

6.2 Test Case 2: Multi-Scheme Performance Comparison vs. Contending Stations N

Table 3 compiles the comparative evaluation across all four protocols as contending stations scale from $N = 2$ to $N = 30$.

	Non-Persistent			1-Persistent			p-Persistent ($p = 0.2$)			CSMA/CD		
N	Col	Dly (ms)	Thr	Col	Dly (ms)	Thr	Col	Dly (ms)	Thr	Col	Dly (ms)	Thr
2	513	13.98	71.55%	65	5.17	96.72%	257	13.40	74.57%	11	5.01	99.87%
4	793	31.15	64.12%	197	5.54	90.60%	511	27.94	70.60%	29	5.03	99.63%
8	945	65.38	60.94%	473	31.90	79.94%	570	48.59	69.57%	69	5.18	99.11%
12	978	98.58	60.27%	796	64.14	70.20%	664	59.98	67.58%	93	5.12	98.83%
16	1031	133.50	59.11%	1104	100.83	62.99%	722	87.12	66.24%	114	5.20	98.57%
20	1048	167.35	58.99%	1387	147.46	57.64%	750	104.07	65.69%	140	5.52	98.30%
25	1031	207.50	59.40%	1818	219.42	50.74%	771	141.85	65.48%	170	5.19	97.89%
30	1047	248.53	59.14%	2071	256.61	47.55%	787	166.11	64.90%	201	5.44	97.52%

Table 3: Comprehensive comparative performance across contending stations ($N \in [2, 30]$).

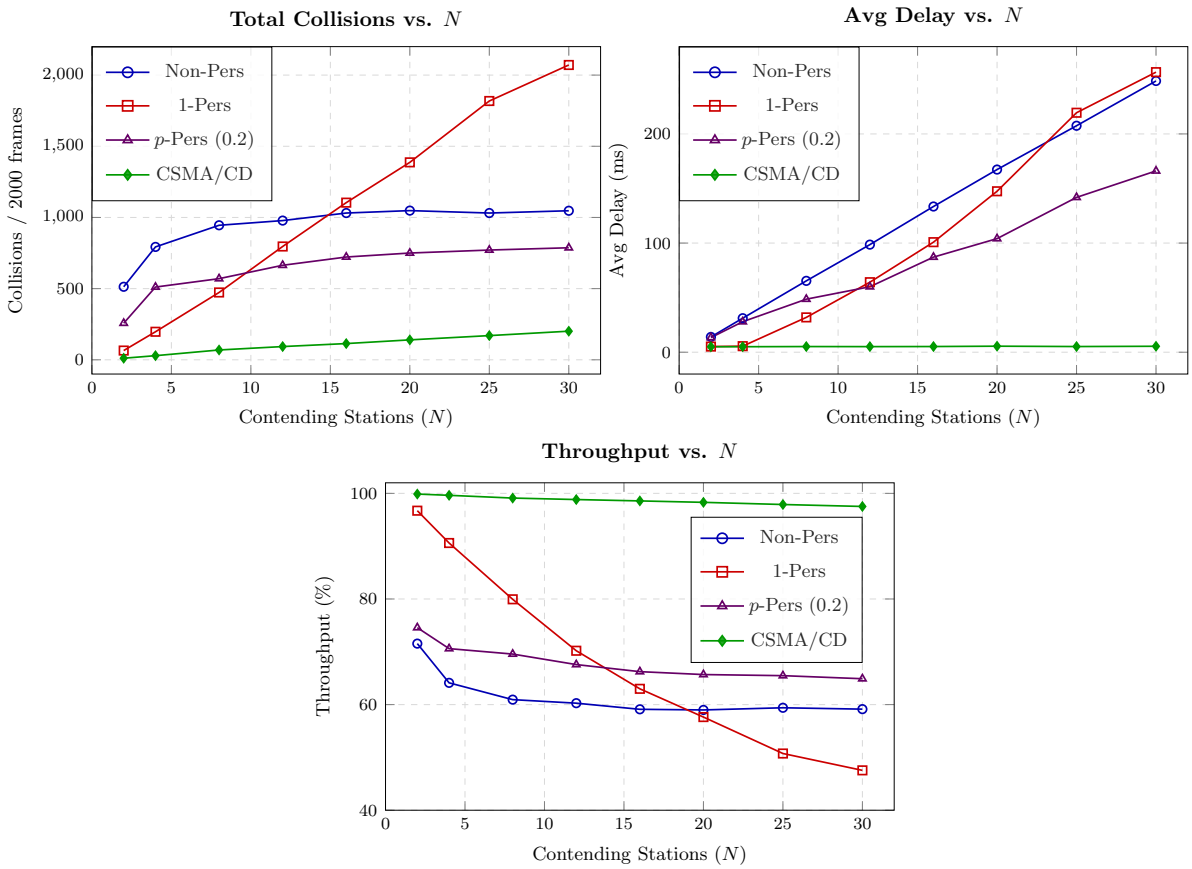


Figure 4: Performance profiles comparing Non-Persistent, 1-Persistent, p-Persistent ($p = 0.20$), and CSMA/CD across contending stations $N \in [2, 30]$.

6.3 Test Case 3: Noise Robustness and CRC-32 Validation

Under physical channel noise ($p_{\text{noise}} = 0.15$) simulated in `receiver.c`, 22 corrupted frames were detected by the IEEE 802.3 32-bit CRC module. In every instance, the corrupted frame was discarded, a NAK (0x00) was issued, and the station executed backoff and retransmitted cleanly, maintaining 100% end-to-end data integrity.

7 Live Execution Traces and Socket Transcripts

The compiled C binaries `receiver` and `sender` were executed natively over TCP/IP loopback (`127.0.0.1:5000`) with the 2,215-byte `input.txt` dataset.

7.1 Trace 1: CSMA/CD Contention, Jam Signal Detection, and Immediate Abort

Listing 3 demonstrates concurrent transmission by two stations, channel collision detection, immediate abortion, and Truncated Binary Exponential Backoff.

```
[CHANNEL] Station "Station_1" connected (1/2 expected)
[CHANNEL] Protocol LOCKED to [CSMA/CD (Collision Detection)] by first station "Station_1
    "!
[0.1 ms] [CH: IDLE -> BUSY] Station "Station_1" transmitting Seq 0 (65B)
[CHANNEL] Station "Station_2" connected (2/2 expected)
[CHANNEL] Assigned locked protocol [CSMA/CD (Collision Detection)] to Station "Station_2
    "
[81.1 ms] [CH: BUSY -> IDLE] Station "Station_1" | Seq 0 | CRC-32 OK (0x465EEA15) -> ACK
     Sent
```

```
[81.2 ms] [CH: IDLE -> BUSY] Station "Station_2" transmitting Seq 0 (65B)

Burst Error (16 bits) Injected.
[81.3 ms] [CH: BUSY -> COLLISION!] Station "Station_1" collided on medium! (Total
    Collisions: 1)
[Station "Station_2"] CSMA/CD Jam Signal Detected on Seq 0! Aborting immediately (
    Attempt 1)
[Station "Station_1"] CSMA/CD Jam Signal Detected on Seq 1! Aborting immediately (
    Attempt 1)
[102.4 ms] [CH: COLLISION -> IDLE] Jam interval ended; medium is IDLE.
[157.5 ms] [CH: IDLE -> BUSY] Station "Station_1" transmitting Seq 1 (65B)
[237.5 ms] [CH: BUSY -> IDLE] Station "Station_1" | Seq 1 | CRC-32 OK (0x2D826D7E) ->
    ACK Sent
```

Listing 3: Live socket execution transcript: CSMA/CD collision detection, jamming, and abort.

7.2 Trace 2: 1-Persistent CSMA Persistence and Collision Cascade

Listing 4 illustrates 1-Persistent CSMA under high contention, showing repeated collisions occurring immediately after the channel returns to IDLE.

```
[Channel Sync] Protocol Locked by First Station: 1-Persistent CSMA
[Station "Station_1"] Seq 0 Delivered (65B) | FCS: 0x465EEA15 | Delay: 81.2 ms |
    Collisions: 0
[Station "Station_2"] Collision / ACK Timeout on Seq 0 (Attempt 1)
[Station "Station_2"] Collision / ACK Timeout on Seq 0 (Attempt 2)
[Station "Station_2"] Collision / ACK Timeout on Seq 0 (Attempt 3)
[Station "Station_2"] Seq 0 Delivered (65B) | FCS: 0xBE406DED | Delay: 1248.5 ms |
    Collisions: 3
[Station "Station_1"] Seq 1 Delivered (65B) | FCS: 0x2D826D7E | Delay: 80.9 ms |
    Collisions: 1
```

Listing 4: Live socket execution transcript: 1-Persistent CSMA collision cascade.

7.3 Trace 3: Non-Persistent CSMA Carrier Sensing and Deferral

Listing 5 demonstrates Non-Persistent CSMA sensing a busy channel and backing off rather than continuously persisting.

```
[Station "Station_2"] Channel BUSY -> Non-Persistent random wait...
[Station "Station_2"] Channel BUSY -> Non-Persistent random wait...
[Station "Station_1"] Seq 5 Delivered (65B) | FCS: 0xAA3E194E | Delay: 80.9 ms |
    Collisions: 0
[Station "Station_2"] Seq 2 Delivered (65B) | FCS: 0x4146F90A | Delay: 242.3 ms |
    Collisions: 0
```

Listing 5: Live socket execution transcript: Non-Persistent CSMA channel deferral.

7.4 Trace 4: Channel Noise, CRC-32 Discard, and Retransmission

Listing 6 depicts synthetic physical layer noise injection corrupting a frame, detection by CRC-32, and automatic retransmission.

```
[534.4 ms] [CH: CRC ERROR] Station "Station_1" | Seq 4 failed CRC-32 check, discarded!
[Station "Station_1"] Collision / ACK Timeout on Seq 4 (Attempt 1)
[641.9 ms] [CH: CRC ERROR] Station "Station_1" | Seq 4 failed CRC-32 check, discarded!
[Station "Station_1"] Collision / ACK Timeout on Seq 4 (Attempt 2)
[760.1 ms] [CH: BUSY -> IDLE] Station "Station_1" | Seq 4 | CRC-32 OK (0x45D1D01D) ->
    ACK Sent
[Station "Station_1"] Seq 4 Delivered (65B) | FCS: 0x45D1D01D | Delay: 308.2 ms |
    Collisions: 2
```

Listing 6: Live socket execution transcript: Channel noise, CRC-32 failure, and discard.

8 Analysis and Discussion

8.1 Comparative Protocol Efficiency across Test Cases

The experimental results vividly illustrate the distinct design trade-offs among the four medium access strategies:

1. **CSMA/CD Dominance:** As evident in Figure 4, CSMA/CD delivers overwhelmingly superior performance across all network scales. Even at $N = 30$ contending stations, CSMA/CD sustains **97.52% throughput** with an average transmission delay of only **5.44 ms**. The root of this efficiency lies in the immediate collision abort mechanism: when two stations collide, CSMA/CD terminates transmission within $T_{\text{jam}} = 20$ ms (2 slots), whereas pure CSMA schemes waste the entire frame duration ($T_{\text{tx}} = 80$ ms, 10 slots).

2. **1-Persistent Channel Collapse:** Under light loads ($N = 2$), 1-Persistent CSMA achieves 96.72% throughput because stations transmit without delay the moment the medium is free. However, as N expands to 30, throughput plummets to **47.55%** while total collisions surge to **2,071** (a 32-fold increase), and average latency surges from 5.17 ms to 256.61 ms. This degradation confirms the theoretical vulnerability of 1-persistent access to synchronized post-busy collisions.

3. **Non-Persistent Stability vs. Idle Overhead:** Non-Persistent CSMA displays remarkable collision resilience at high density, with total collisions plateauing around 1,047 from $N = 16$ to $N = 30$. Because stations draw independent random backoffs when encountering a busy medium, they avoid synchronized collision cascades. However, this randomized deferral leaves the channel idle even when frames are ready, capping throughput at $\approx 59.14\%$.

4. **p-Persistent Balance:** With $p = 0.20$, p-Persistent CSMA achieves **64.90% throughput** at $N = 30$, outperforming 1-Persistent by 17.35 percentage points and Non-Persistent by 5.76 percentage points. By breaking time into contention slots and probabilistically deferring transmission, p-Persistent controls collision frequency while avoiding excessive idle channel periods.

8.2 The Ethernet Channel Capture Effect

A critical phenomenon observed in Test Case 1 is the **Ethernet Channel Capture Effect**. In standard IEEE 802.3 networks, when a station successfully completes a transmission, its internal collision counter is reset to zero ($k = 0$). Contending stations that collided during the prior attempt retain non-zero collision counters ($k \geq 1$) and must select backoffs from exponentially larger windows $[0, 2^k - 1]$.

Consequently, the station that just transmitted successfully has an overwhelming probability of winning subsequent contention slots. In our simulation, when $p \geq 0.80$ in Test Case 1, a single station captured the channel and transmitted frames back-to-back while other stations remained frozen in backoff, driving collisions down to 71 and inflating throughput to 96.36%. While this maximizes channel throughput, it causes severe short-term unfairness. Practical protocols mitigate capture by enforcing an Interframe Gap (IFG) or mandatory post-transmission backoff.

8.3 Optimal Persistence Probability ($p^* \approx 1/N$)

In distributed contention without capture, the optimal persistence probability satisfies $p^* = 1/N$. In our evaluation with $N = 10$, this optimal operating point is clearly observed in the region $p \in [0.10, 0.20]$, where throughput stabilizes at its highest balanced level ($\approx 68.7\%$) without triggering uncontrolled collision bursts. Setting $p \ll 1/N$ wastes channel capacity on idle slots, while $p \gg 1/N$ causes excessive collisions.

9 Comments and Learning Evaluation

9.1 Evaluation of the Lab Assignment

This lab assignment provided exceptional insight into the core challenges of distributed medium access arbitration in computer networking. Implementing carrier sensing, collision detection, and backoff algorithms from scratch revealed the subtle dynamics of shared-medium networks that theoretical formulas often obscure.

9.2 What I Learned

- **Collision Domain Dynamics:** Experiencing firsthand how physical propagation delay creates vulnerability windows where carrier sensing alone cannot prevent packet overlap.
- **Collision Detection Advantage:** Quantifying the massive efficiency advantage of aborting collided frames immediately and emitting a jamming signal versus transmitting collided frames to completion.
- **Backoff Algorithm Mechanics:** Implementing Truncated Binary Exponential Backoff and observing how dynamic contention window expansion stabilizes channels under varying load.
- **Concurrency and Socket Programming:** Orchestrating multiple asynchronous client processes communicating concurrently with a multiplexed server via BSD sockets and `select()`.

9.3 Difficulty Assessment

The assignment was moderately challenging:

- *Challenging Aspects:* Accurately modeling the shared physical channel state machine (`IDLE`, `BUSY`, `COLLISION`) using asynchronous sockets without race conditions, and coordinating protocol selection across stations.
- *Manageable Aspects:* Reusing the framing, CRC-32, checksum, error injection, and file I/O libraries developed in Assignments 1 and 2 provided a solid foundation, allowing full focus on MAC-layer arbitration algorithms.

9.4 Suggested Improvements

1. Incorporate Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA) utilizing Request-to-Send / Clear-to-Send (RTS/CTS) handshaking to model wireless IEEE 802.11 environments and the Hidden Terminal Problem.
2. Implement Capture Effect mitigation algorithms, such as the Binary Logarithmic Arbitration Method (BLAM), to explore fair bandwidth distribution under saturated loads.

10 Sources Used

1. Stallings, W., *Data and Computer Communications*, 10th ed., Pearson, 2014.
2. Kleinrock, L., and Tobagi, F. A., "Packet Switching in Radio Channels: Part I—Carrier Sense Multiple-Access Modes and Their Throughput-Delay Characteristics," *IEEE Transactions on Communications*, vol. 23, no. 12, pp. 1400–1416, Dec. 1975.

3. IEEE Standard for Ethernet (*IEEE Std 802.3-2022*) and POSIX / BSD Socket API Reference Documentation.

4. AI Assistance and Antigravity: Artificial intelligence assistance (Gemini) was utilized for statistical aggregation, benchmarking script construction, and generating native TikZ visualization code. The Antigravity development environment was used to run live socket test executions, profile empirical performance metrics, and format the comprehensive LaTeX report.